PROJETO INTEGRADOR V · SPRINT ATUAL

Da revisão da arquitetura à primeira API funcionando

Atualização objetiva: decisões, ajustes, testes realizados e próximos passos.

ARQUITETURA

W5100 + NestJS + MongoDB

ENTREGA ATUAL

POST /leituras em memória

PRÓXIMA TAREFA

DTO + validação global

VISÃO DE PROJETO

Onde estamos agora

Monitorar níveis altos e baixos da água; nesta sprint, o foco foi a base de comunicação e a API.

Base testada

SENSORIAMENTO

HC-SR04 com filtragem por média; DHT e RTC em testes anteriores.

Em implementação

BACKEND

NestJS criado; resource leituras e POST mínimo operacionais.

Ainda pendentes

INTEGRAÇÕES

W5100 real, MongoDB, dashboard, alertas e processamento preditivo.

FOCO DA ETAPA: garantir que o Arduino consiga enviar um JSON padronizado para uma API confiável.

DECISÕES

As mudanças que definiram o novo caminho

A proposta anterior foi revisada para simplificar a comunicação e apoiar o aprendizado do backend.

DECISÃO	ANTES	AGORA
Conectividade	ESP12F / Wi-Fi	**Ethernet Shield W5100 / cabo**
Backend	Estrutura preliminar em Python	**Recriação do zero em NestJS + TypeScript**
Banco de dados	SQLite / PostgreSQL	**MongoDB (Prisma planejado)**
Testes de API	Thunder Client	**Bruno, integrado ao VS Code**

Critério arquitetural: Arduino envia dados para a API — não acessa o MongoDB diretamente.

INTEGRAÇÃO

Novo fluxo de dados planejado

O contrato de comunicação separa a coleta embarcada do processamento e armazenamento.



Ponto-chave O backend recebe e valida; somente ele ficará responsável pelas credenciais e pelo acesso ao banco.

CONTRATO V1

O que o Arduino vai enviar

Definição concluída antes da montagem final do circuito, com dados simulados para os primeiros testes.

JSON / POST /leituras

```
{
  "versao": 1,
  "idDispositivo": "arduino-mega-01",
  "idLeitura": "arduino-mega-01-001",
  "timestampRtc": "2026-10-22T16:00:00-03:00",
  "distanciaCm": 35.7,
  "nivelUltrassonicoCm": 14.3,
  "nivelResistivoCm": 3.2,
  "umidadePct": 65.0
}
```

OBRIGATÓRIOS

versao · idDispositivo · idLeitura · timestampRtc

OPCIONAIS

distanciaCm · nivelUltrassonicoCm
nivelResistivoCm · umidadePct

REGRAS

Maquete: envio a cada 5 segundos.
Uso real: 1 hora (a validar).
Falha em sensor: enviar demais dados.

ENTREGAS REALIZADAS

Backend NestJS: o que já programamos

Estrutura gerada pelo CLI e comunicação HTTP mínima, ainda sem banco de dados.

ESTRUTURA GERADA

```
backend/src/
├─ main.ts
├─ app.module.ts
└─ leituras/
   ├─ leituras.module.ts
   ├─ leituras.controller.ts
   ├─ leituras.service.ts
   └─ dto/
      └─ create-leitura.dto.ts
```

Os dados em memória são perdidos ao reiniciar o servidor.

FUNCIONALIDADES DA ETAPA

- **ConfigModule + .env**
  Porta e endereço configuráveis (HOST / PORT).
- **Resource leituras**
  Controller, service e módulo criados pelo CLI.
- **POST /leituras**
  Recebe JSON; acrescenta recebidoEm; guarda em array.
- **Bruno**
  POST testado após correção do módulo.

DADOS COMPLEMENTARES

Integrações preparadas, mas fora desta entrega

A pesquisa meteorológica e histórica continua relevante sem competir com o foco atual do backend.

PROVA DE CONCEITO

Open-Meteo

Consulta GET por latitude e longitude; versão Python recebeu 72 previsões horárias (3 dias).

Próximo uso: Reimplementar integração no NestJS.

PESQUISA HISTÓRICA

ANA / HidroWeb

Encantado (RS): cota e vazão — estação 86720000; chuva — estação 2951010.

Objetivo: Estudar cheias e períodos de níveis baixos.

Integração preditiva / FPGA: horizonte futuro, ainda sem implementação nesta sprint.

PRÓXIMOS PASSOS

O que programaremos a seguir

Prioridade: garantir qualidade da entrada antes de integrar sensores físicos ou persistência.

T01.4

Confirmar GET /leituras

Verificar retorno das leituras armazenadas em memória pelo Bruno.

T01.5

DTO + ValidationPipe

Aplicar campos obrigatórios, opcionais e regras do contrato V1.

BANCO

Prisma + MongoDB

Persistir medições, criar índice único e tratar retransmissões.

HARDWARE

Ethernet Shield W5100

Montar o circuito e testar POST real do Arduino para a API.

META DA PRÓXIMA ETAPA

API com JSON validado, pronta para receber leituras reais e evoluir para persistência.